

# AWS Self-Learning & Hands-On Implementation

## Topics Covered

1. Amazon SQS — Simple Queue Service
2. Amazon MQ
3. AWS Secrets Manager

## Important Comparison

| Area                            | SQS             | Amazon MQ                                                  | Secrets Manager    |
|---------------------------------|-----------------|------------------------------------------------------------|--------------------|
| Main purpose                    | Message queuing | Message brokering                                          | Secret management  |
| Primary object                  | Queue           | Broker                                                     | Secret             |
| Producer/Consumer               | Yes             | Yes                                                        | No                 |
| Stores messages                 | Yes             | Yes                                                        | No                 |
| Stores passwords                | No              | Credentials for broker access, but not its primary purpose | Yes                |
| Decoupling                      | Yes             | Yes                                                        | No                 |
| Traditional messaging protocols | Limited         | Yes                                                        | Not applicable     |
| IAM                             | Yes             | Used with AWS resources/access patterns                    | Very important     |
| Encryption                      | Yes             | Yes                                                        | Yes                |
| Rotation                        | Not applicable  | Not primary feature                                        | Yes                |
| Main DevOps use                 | Async workloads | Legacy/enterprise messaging                                | Secure credentials |

## 1. Amazon SQS — Simple Queue Service

### 1.1 Introduction to Amazon SQS

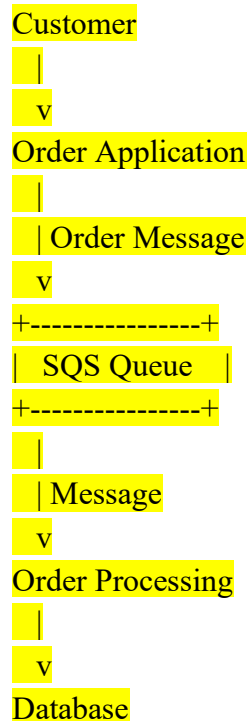
Amazon Simple Queue Service (Amazon SQS) is a fully managed message queuing service provided by AWS.

It allows applications and services to communicate **asynchronously**.

The main purpose of SQS is to **decouple application components**.

#### Simple example

Suppose we have an e-commerce application:



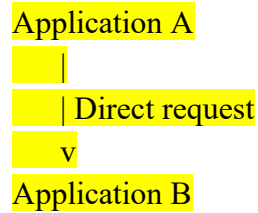
The Order Application does not need to directly communicate with the Order Processing application.

Instead, it places the order message into SQS.

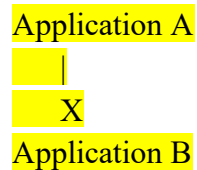
The consumer can process the message whenever it is ready.

## 1.2 Why SQS is Required

Without a queue:

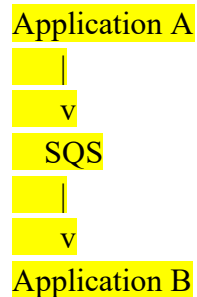


If Application B is unavailable:



This can cause failures.

With SQS:



If Application B is temporarily unavailable, messages can remain in the queue until they are processed.

### Main benefits

- Application decoupling
- Asynchronous processing
- Reliability
- Scalability

- Fault isolation
- Buffering
- Retry handling

### **1.3 Important SQS Terminology**

#### **Producer**

The application that sends messages.

Producer → SQS

#### **Queue**

The SQS resource that holds messages.

#### **Message**

The actual data being transferred.

Example:

OrderID=1001

Product=Laptop

Quantity=1

#### **Consumer**

The application that retrieves and processes messages.

SQS → Consumer

#### **Visibility Timeout**

The period during which a message becomes temporarily invisible after a consumer receives it.

#### **Dead-Letter Queue**

A separate queue used to hold messages that could not be successfully processed after repeated attempts.

## Message Retention

The amount of time SQS keeps a message if it isn't deleted.

## 1.4 SQS Queue Types

### Standard Queue

Standard queues are designed for high throughput and scalability.

Characteristics:

- At-least-once delivery
- Very high throughput
- Best-effort ordering

Example:

Message 1

Message 2

Message 3

The consumer may not necessarily receive them in exactly that order.

### FIFO Queue

FIFO means:

First In, First Out

FIFO queues are used when ordering is important.

Example:

Message 1

↓

Message 2

↓

Message 3

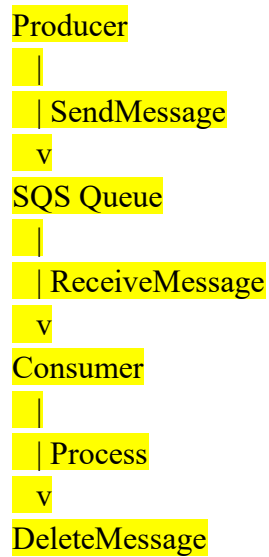
Processing should preserve the required ordering.

### Use case

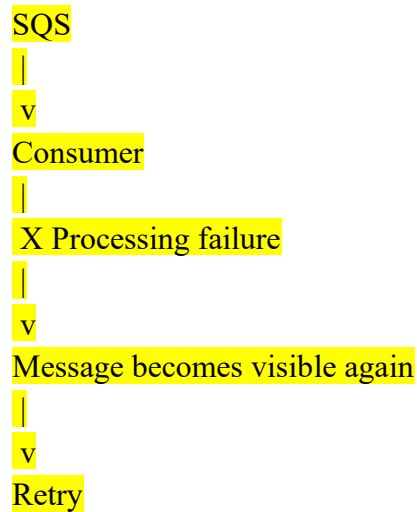
Banking transactions or order processing where sequence matters.

## 1.5 SQS Message Lifecycle

The lifecycle is:



If processing fails:



After repeated failures:

Main Queue



v

Retry attempts



v

Dead-Letter Queue

## 1.6 SQS Security

SQS can integrate with AWS security mechanisms such as IAM.

IAM permissions determine who can perform actions such as:

SendMessage

ReceiveMessage

DeleteMessage

GetQueueAttributes

You should understand:

IAM



| Permission



SQS Queue

## 1.7 SQS Real-Time Use Cases

### E-commerce

Order Application

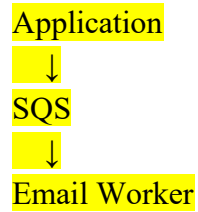


SQS

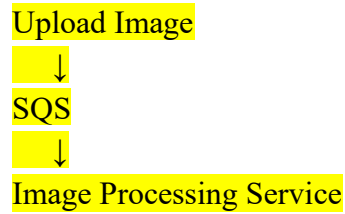


Order Processing

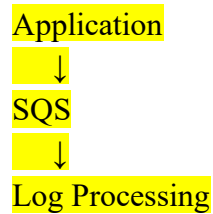
## Email processing



## Image processing



## Log processing



## Important SQS Components

| Component          | Meaning                              |
|--------------------|--------------------------------------|
| Producer           | Sends message                        |
| Queue              | Stores message                       |
| Message            | Data being transferred               |
| Consumer           | Receives/processes message           |
| Visibility Timeout | Temporarily hides received message   |
| DLQ                | Stores messages that repeatedly fail |
| Retention Period   | How long messages are retained       |

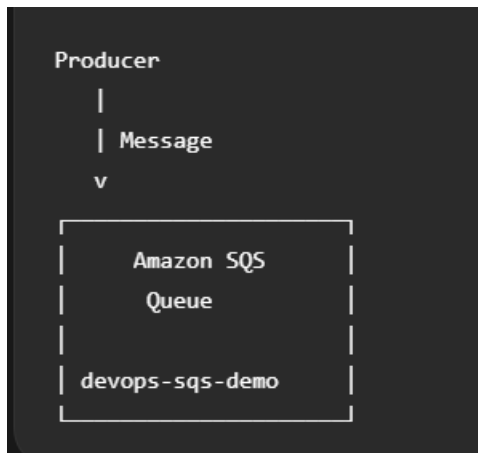
## Task 1 — Create an SQS Queue

### Objective

Create a **Standard SQS queue** that will later be used to:

- Send messages
- Receive messages
- Test visibility timeout
- Configure a Dead-Letter Queue

### Architecture for this task



### Step 1 — Open SQS

Go to the AWS Console.

Search for:

**SQS**

Open:

**Amazon Simple Queue Service**

Make sure you're working in the AWS region you want to use for the lab. Since you've commonly been using **US East (N. Virginia)** — **us-east-1**, we'll use that consistently.

## Step 2 — Create the queue

Click:

### Create queue

You'll see the queue configuration page.

### Queue type

Select:

#### Standard

For now, don't select FIFO.

### Queue name

Use:

devops-sqs-demo

## Step 3 — Queue configuration

For this first task, **don't change the advanced settings.**

We want a simple queue first.

You'll see settings such as:

- Visibility timeout
- Message retention period
- Delivery delay
- Maximum message size
- Receive message wait time
- Encryption
- Access policy

We'll learn these separately during the following tasks.

So for now, keep the defaults unless AWS requires a value.

## Step 4 — Create

Scroll to the bottom and click:

### Create queue

You should now see:

devops-sqs-demo

with the queue status available/active.

## Step 5 — Verify the queue

Open the queue.

You should be able to see information such as:

Queue name

Queue URL

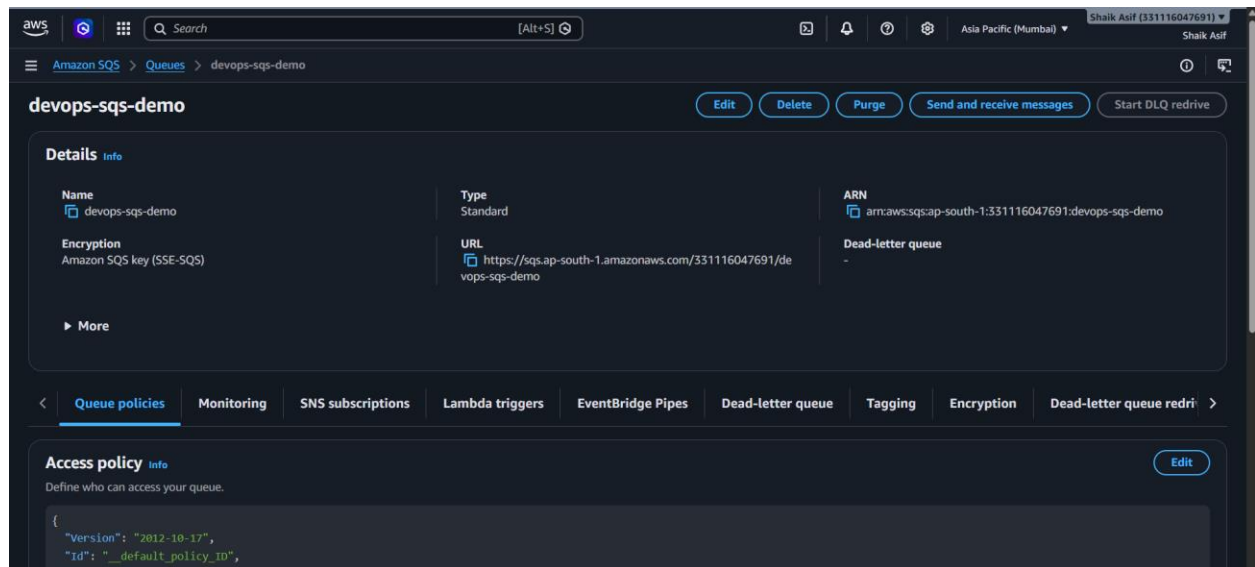
Queue ARN

Queue type

Configuration

The **Queue ARN** is the unique AWS identifier for the queue.

The **Queue URL** is used by applications/tools to interact with the queue.



### Observation:

The SQS queue was successfully created and became available for sending and receiving messages. AWS automatically provided a unique Queue URL and Queue ARN for the queue.

### Result:

The devops-sqs-demo Standard SQS queue was successfully created.

### Conclusion:

Amazon SQS provides a managed queue that can be used to decouple application components and enable asynchronous message processing.

## Task 2 — Send a Message to the SQS Queue

## Steps Used:

1. Logged in to the **AWS Management Console**.
2. Selected the **Mumbai (ap-south-1)** AWS Region.
3. Opened the **Amazon SQS** service.
4. Navigated to **Queues**.
5. Opened the previously created queue **devops-sqs-demo**.
6. Selected **Send and receive messages**.
7. Entered the message:  
Hello from Asifshaik
8. Clicked **Send message**.
9. Verified that the message was successfully sent to the SQS queue.
10. Verified that the message was available in the queue for a consumer to retrieve.

The screenshot shows the AWS Management Console interface for the 'devops-sqs-demo' queue. The page title is 'Send and receive messages'. Below the title, there is a description: 'Use this page to send, retrieve and view messages, helping you experiment with various queue features.' The main form has three sections: 'Send message' (with an 'Info' link), 'Message body' (with a text input field containing 'Hello from Asifshaik'), 'Message group ID' (with a text input field and a description: 'A group identifier for the message to allow fair processing across message groups in a standard queue. Message group ID must be 1 to 128 characters. Valid characters are a-z, A-Z, 0-9, and punctuation (!\"#\$%&'()\*+,-./:;<=>?@[\^\_`{|}~).'), and 'Delivery delay' (with a dropdown menu set to '0' and a unit selector set to 'Seconds'). At the bottom right, there are two buttons: 'Clear content' and 'Send message'.

aws [Search] [Alt+S] Asia Pacific (Mumbai) Shaik Asif (33116047691) Shaik Asif

Amazon SQS > Queues > devops-sqs-demo > Send and receive messages

### Send and receive messages

Use this page to send, retrieve and view messages, helping you experiment with various queue features.

#### Send message [Info](#)

**Message body**  
Enter the message to send to the queue.

Hello from Asifshaik

**Message group ID** - optional, new [Info](#)  
A group identifier for the message to allow fair processing across message groups in a standard queue.

Message group ID must be 1 to 128 characters. Valid characters are a-z, A-Z, 0-9, and punctuation (!\"#\$%&'()\*+,-./:;<=>?@[\^\_`{|}~).

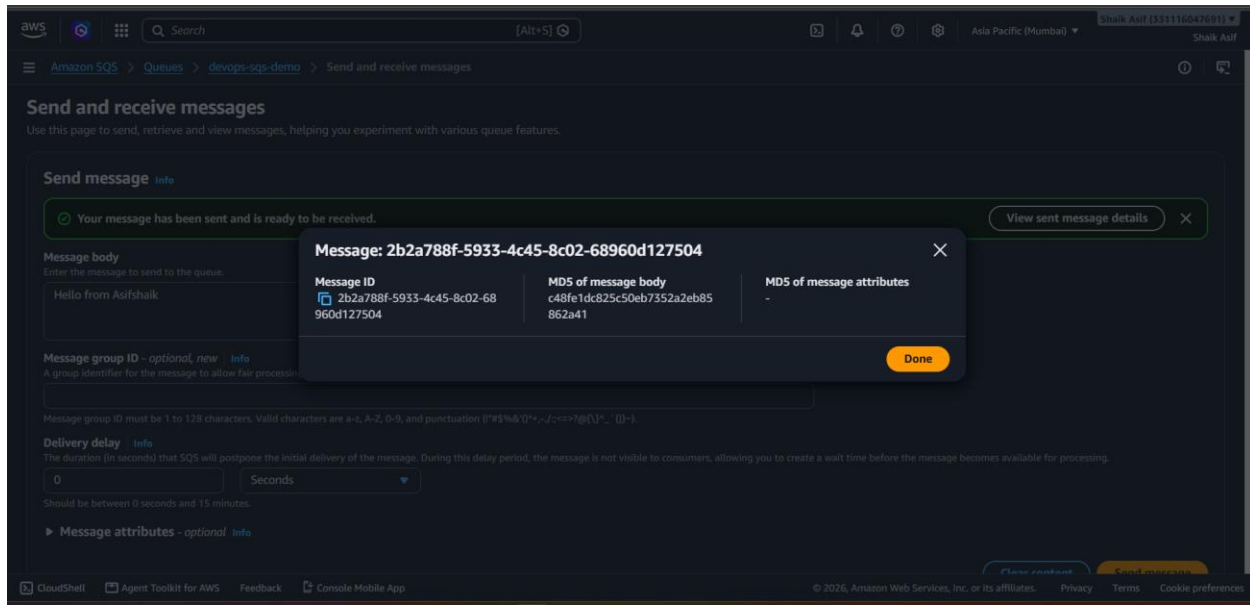
**Delivery delay** [Info](#)  
The duration (in seconds) that SQS will postpone the initial delivery of the message. During this delay period, the message is not visible to consumers, allowing you to create a wait time before the message becomes available for processing.

0 Seconds

Should be between 0 seconds and 15 minutes.

► **Message attributes** - optional [Info](#)

[Clear content](#) [Send message](#)



## Observation

The message was successfully sent and stored in the **devops-sqs-demo** queue. The message remains in the queue until a consumer receives and processes it.

## Result

The message was successfully sent to the Amazon SQS Standard Queue in the Mumbai (ap-south-1) region.

## Conclusion

Amazon SQS allows producers to send messages to a queue without directly communicating with consumers, providing **asynchronous communication and application decoupling**.

## Task 3 — Receive and Delete the SQS Message

### Steps Used:

- Logged in to the AWS Management Console.
- Selected the **Mumbai (ap-south-1)** region.
- Opened **Amazon SQS**.
- Navigated to **Queues**.
- Opened the **devops-sqs-demo** queue.
- Selected **Send and receive messages**.

- Clicked **Poll for messages** to retrieve the message.
- Verified the message body **Hello from asifshaik**
- Selected the received message.
- Deleted the message from the queue.
- Polled the queue again and verified that the processed message was no longer available.

**Delivery delay** Info  
The duration (in seconds) that SQS will postpone the initial delivery of the message. During this delay period, the message is not visible to consumers, allowing you to create a wait time before the message becomes available for processing.

0 Seconds

Should be between 0 seconds and 15 minutes.

► **Message attributes** - optional Info

Clear content Send message

**Receive messages** Info

Messages available: 0

Polling duration: 30

Maximum message count: 10

Polling progress: ✔ SUCCESS  
0 receives/second

Messages (1)

Search messages

| ID                                   | Sent                   | Size     | Receive count |
|--------------------------------------|------------------------|----------|---------------|
| 2b2a788f-5933-4c45-8c02-68960d127504 | 2026-09-22T14:55+05:30 | 20 bytes | 1             |

View details Delete

**Received message: 2b2a788f-5933-4c45-8c02-68960d127504**

Body Attributes Details

Hello from Asifshaik

Done

Clear content Send message

Stop polling Poll for messages

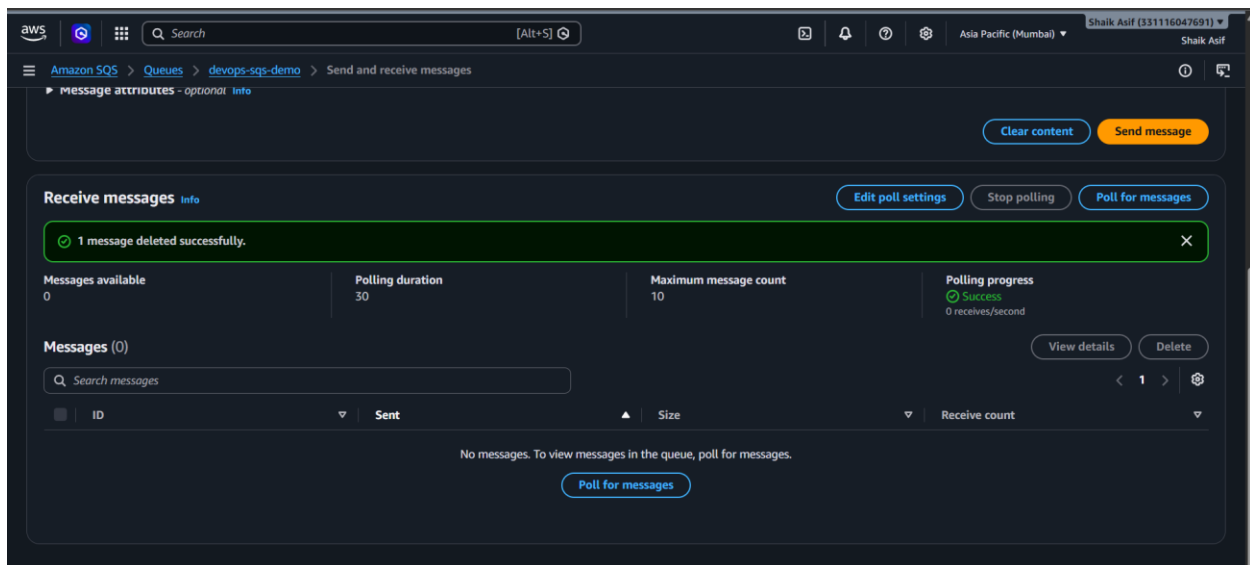
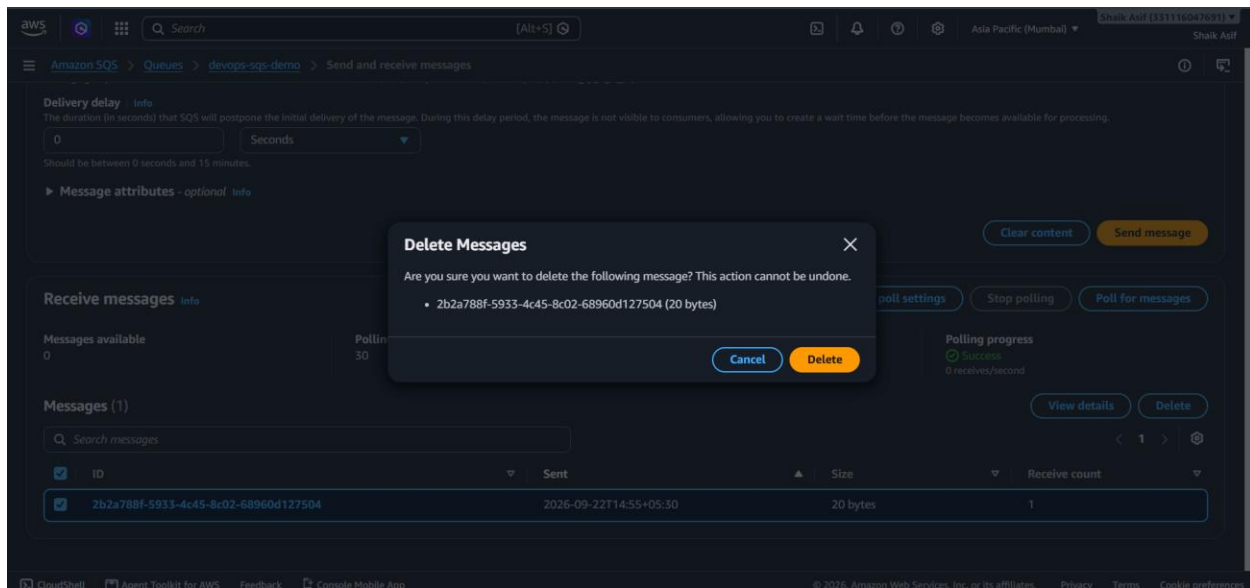
Messages available: 0

Messages (1)

Search messages

| ID                                   | Sent                   | Size     | Receive count |
|--------------------------------------|------------------------|----------|---------------|
| 2b2a788f-5933-4c45-8c02-68960d127504 | 2026-09-22T14:55+05:30 | 20 bytes | 1             |

View details Delete



## Observation

The message sent in Task 2 was successfully retrieved by the consumer. After processing, the message was deleted from the queue.

## Result

**The SQS message was successfully received and deleted from the devops-sqs-demo queue.**

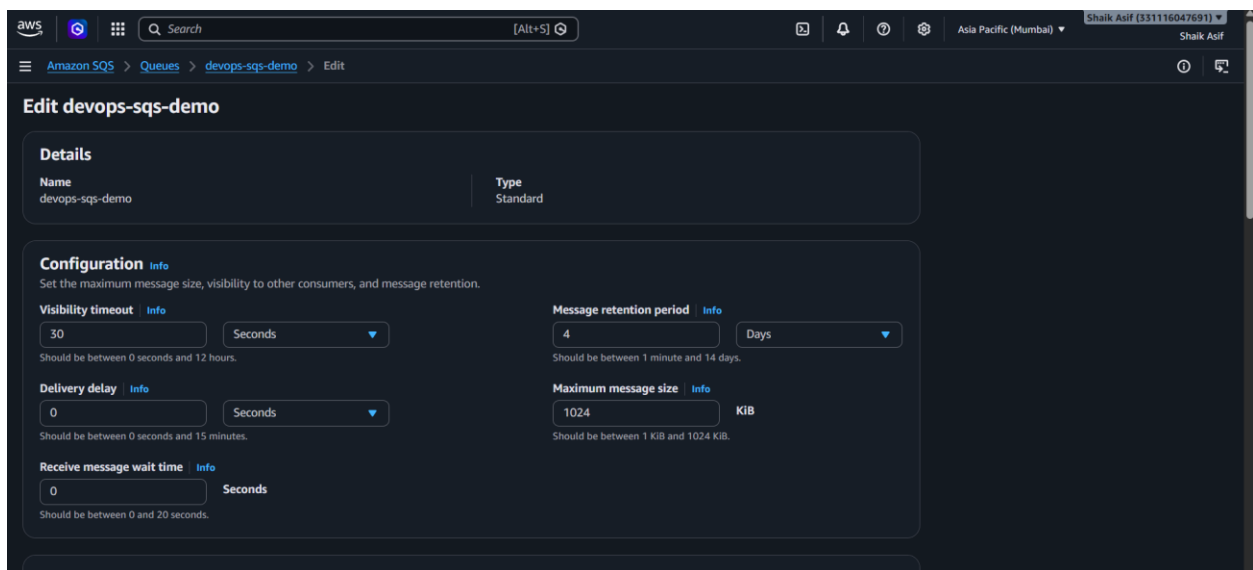
## Conclusion

SQS allows consumers to retrieve messages asynchronously. After successful processing, the consumer deletes the message so that it is not processed again.

## Task 4 — Test SQS Visibility Timeout

### Steps Used:

- Logged in to the AWS Management Console.
- Selected the **Mumbai (ap-south-1)** region.
- Opened Amazon SQS.
- Opened the devops-sqs-demo queue.
- Edited the queue configuration.
- Set the **Visibility Timeout** to **30 seconds**.
- Saved the queue configuration.
- Sent a new message named Visibility timeout test message.
- Polled the queue and received the message.
- Did not delete the message immediately.
- Observed that the received message became temporarily invisible.
- Waited for the visibility timeout to expire.
- Polled the queue again and verified that the message became available again.
- Deleted the message after completing the test.



aws [Search] [Alt+S] Asia Pacific (Mumbai) Shaik Asif (331116047691)

Amazon SQS > Queues > devops-sqs-demo > Send and receive messages

## Send and receive messages

Use this page to send, retrieve and view messages, helping you experiment with various queue features.

### Send message [Info](#)

**Message body**  
Enter the message to send to the queue.

Visibility timeout test message

**Message group ID - optional, new** [Info](#)  
A group identifier for the message to allow fair processing across message groups in a standard queue.

Message group ID must be 1 to 128 characters. Valid characters are a-z, A-Z, 0-9, and punctuation ([\*#\$%&'()\*+,-./:;<=>?@[\]^\_`{|}~]).

**Delivery delay** [Info](#)  
The duration (in seconds) that SQS will postpone the initial delivery of the message. During this delay period, the message is not visible to consumers, allowing you to create a wait time before the message becomes available for processing.

0 Seconds

Should be between 0 seconds and 15 minutes.

► **Message attributes - optional** [Info](#)

[Clear content](#) [Send message](#)

aws [Search] [Alt+S] Asia Pacific (Mumbai) Shaik Asif (331116047691)

Amazon SQS > Queues > devops-sqs-demo > Send and receive messages

The duration (in seconds) that SQS will postpone the initial delivery of the message. During this delay period, the message is not visible to consumers, allowing you to create a wait time before the message becomes available for processing.

0 Seconds

Should be between 0 seconds and 15 minutes.

► **Message attributes - optional** [Info](#)

**Received message: cb58e5f5-9d1c-4a4e-b008-3fee8febb72**

[Body](#) [Attributes](#) [Details](#)

Visibility timeout test message

[Done](#)

[Clear content](#) [Send message](#)

[Stop polling](#) [Poll for messages](#)

ing progress  
0.0000  
messages/second

[View details](#) [Delete](#)

1

**Receive messages [Info](#)**

Messages available  
2

**Messages (2)**

Search messages

| <input type="checkbox"/> | ID                                   | Sent                   | Size     | Receive count |
|--------------------------|--------------------------------------|------------------------|----------|---------------|
| <input type="checkbox"/> | cb58e5f5-9d1c-4a4e-b008-3fee8febb72  | 2026-09-22T15:13+05:30 | 31 bytes | 2             |
| <input type="checkbox"/> | 56947c90-bce4-4c9e-95da-6d85249292af | 2026-09-22T15:13+05:30 | 31 bytes | 1             |

## Observation

After the consumer received the message, it was temporarily hidden because of the configured visibility timeout. Since the message was not deleted, it became available again after the timeout expired.

## Result

**Visibility Timeout was successfully tested on the devops-sqs-demo queue.**

## Conclusion

Visibility Timeout prevents multiple consumers from processing the same message simultaneously while a consumer is working on it. If processing fails and the message is not deleted, SQS makes the message available again after the visibility timeout.

## Task 5 — Create a Dead-Letter Queue (DLQ)

Steps Used :

- Logged in to the AWS Management Console.
- Selected the **Mumbai (ap-south-1)** region.
- Opened Amazon SQS.
- Created a Standard queue named `devops-sqs-dlq`.
- Opened the main queue `devops-sqs-demo`.
- Edited the queue configuration.
- Configured `devops-sqs-dlq` as the Dead-Letter Queue.
- Set the maximum receive count to **3**.
- Saved the redrive policy configuration.
- Verified that the main queue was configured with the DLQ.

The screenshot shows the 'Create queue' page in the AWS Management Console. The breadcrumb navigation at the top indicates the path: Amazon SQS > Queues > Create queue. The page is titled 'Create queue' and is divided into three main sections: Details, Name, and Configuration.

**Details**

Type

Choose the queue type for your application or cloud infrastructure.

☒ **Standard** [Info](#)

At-least-once delivery, message ordering isn't preserved

- At-least once delivery
- Best-effort ordering

☐ **FIFO** [Info](#)

First-in-first-out delivery, message ordering is preserved

- First-in-first-out delivery
- Exactly-once processing

**Warning:** You can't change the queue type after you create a queue.

**Name**

A queue name is case-sensitive and can have up to 80 characters. You can use alphanumeric characters, hyphens (-), and underscores (\_).

**Configuration** [Info](#)

Set the maximum message size, visibility to other consumers, and message retention.

**Visibility timeout** [Info](#)

**Message retention period** [Info](#)

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Amazon SQS > Queues > devops-sqs-dlq

Queue devops-sqs-dlq created successfully

You can now send and receive messages.

devops-sqs-dlq

Edit

Delete

Purge

Send and receive messages

Start DLQ redrive

Details

info

Name

devops-sqs-dlq

Type

Standard

ARN

arn:aws:sqs:ap-south-1:331116047691:devops-sqs-dlq

Encryption

Amazon SQS key (SSE-SQS)

URL

https://sqs.ap-south-1.amazonaws.com/331116047691/devops-sqs-dlq

Dead-letter queue

-

More

Queue policies

Monitoring

SNS subscriptions

Lambda triggers

EventBridge Pipes

Dead-letter queue

Tagging

Encryption

Dead-letter queue redrive

Access policy

info

Edit

Define who can access your queue.

aws

Search

[Alt+S]

Asia Pacific (Mumbai)

Shaik Asif (331116047691)

Amazon SQS > Queues > devops-sqs-demo > Edit

Policy generator

Redrive allow policy

Optional

info

Identify which source queues can use this queue as the dead-letter queue.

Select which source queues can use this queue as the dead-letter queue.

Disabled

Enabled

Dead-letter queue

Optional

info

Send undeliverable messages to a dead-letter queue.

Set this queue to receive undeliverable messages.

Disabled

Enabled

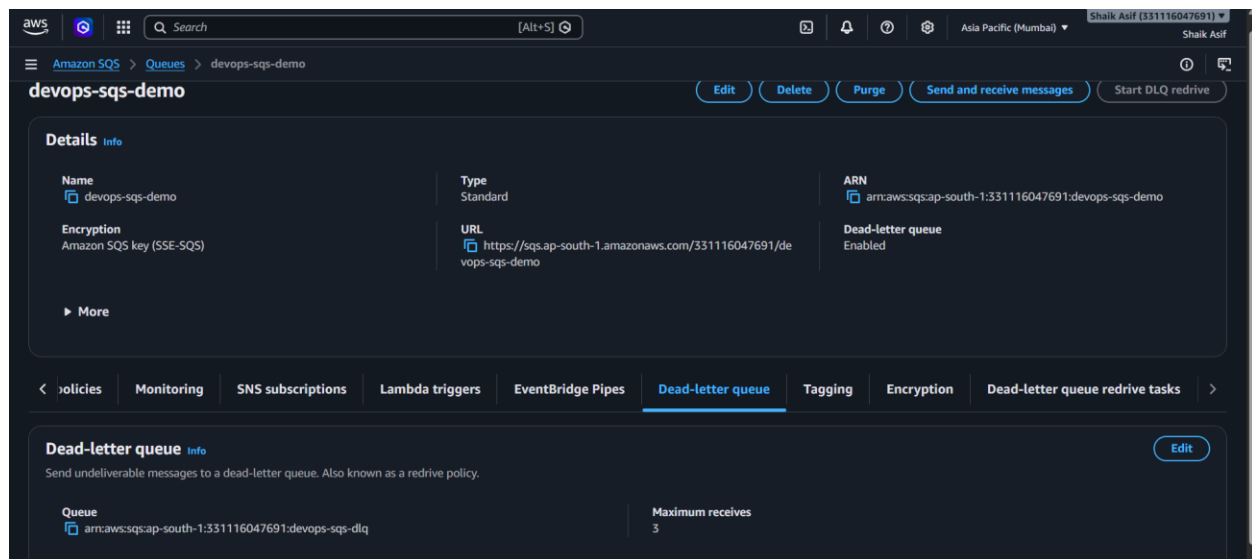
Choose queue

arn:aws:sqs:ap-south-1:331116047691:devops-sqs-dlq

Maximum receives

3

Should be between 1 and 1000



## Observation

The devops-sqs-demo queue was configured with devops-sqs-dlq as its Dead-Letter Queue. Messages that repeatedly fail processing can be redirected to the DLQ after reaching the configured receive limit.

## Result

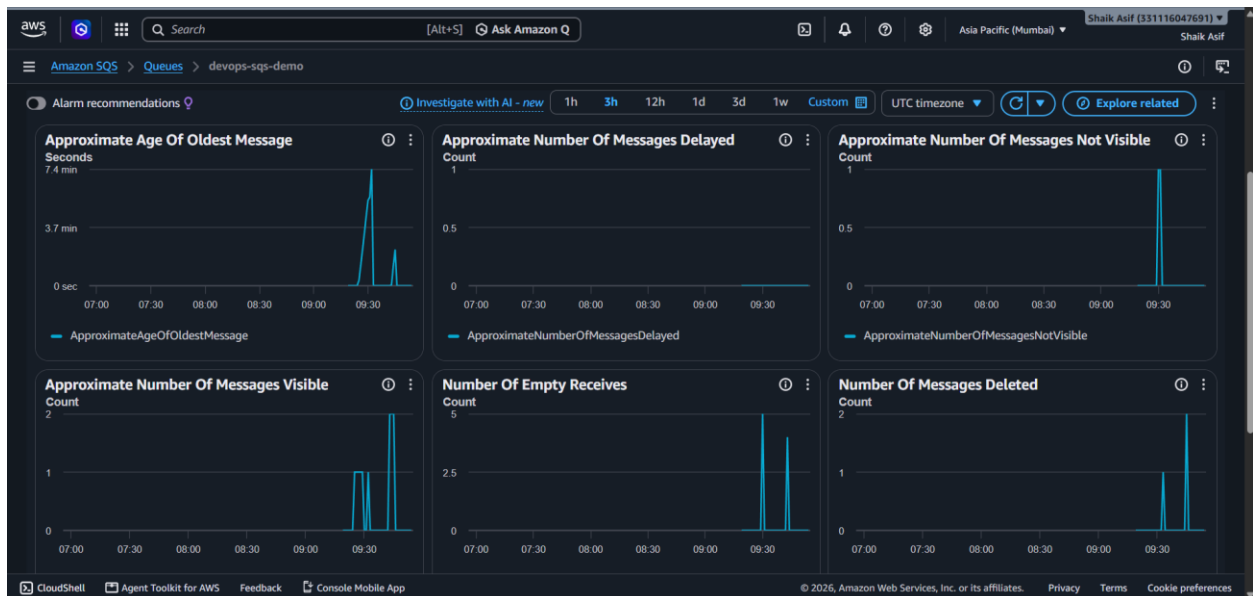
**The Dead-Letter Queue was successfully created and configured for the main SQS queue.**

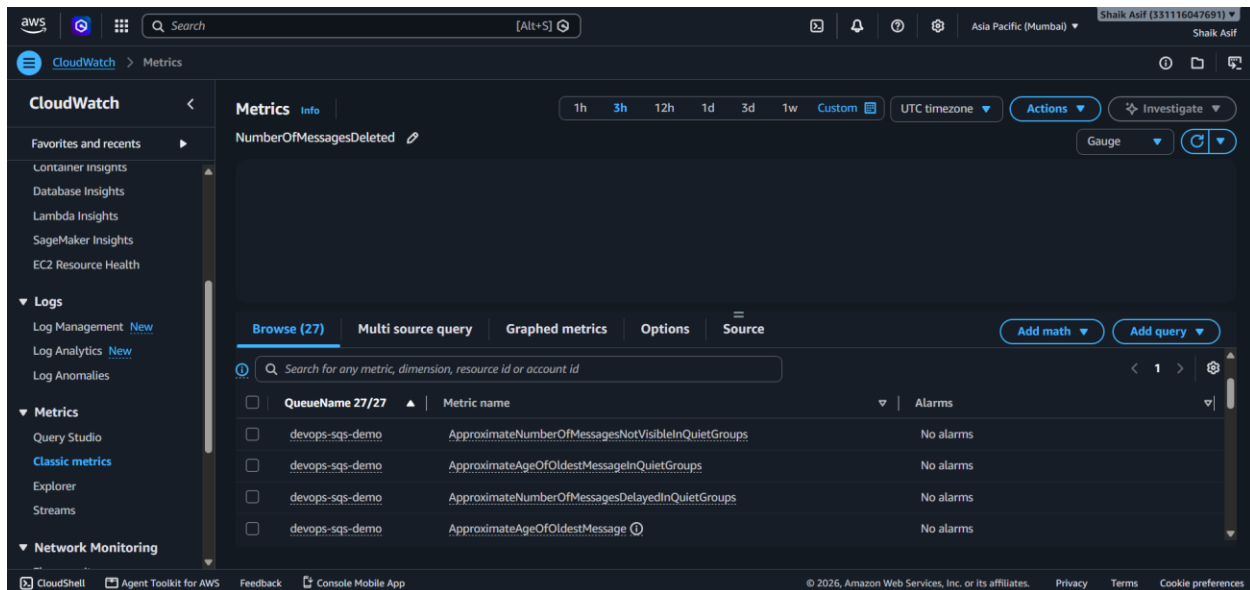
## Conclusion

A DLQ helps isolate messages that cannot be successfully processed. This prevents repeatedly failing messages from continuously interfering with normal message processing and allows DevOps teams to investigate failed messages separately.

## Task 6 — Monitor SQS Using CloudWatch

- Logged in to the AWS Management Console.
- Selected the **Mumbai (ap-south-1)** region.
- Opened Amazon SQS.
- Opened the devops-sqs-demo queue.
- Navigated to the monitoring/metrics section.
- Reviewed SQS queue metrics.
- Checked messages sent, received and deleted.
- Reviewed visible and not-visible message counts.
- Reviewed the approximate age of the oldest message.
- Opened Amazon CloudWatch to review SQS metrics.





## Observation

SQS provides monitoring metrics that help identify message backlogs, processing activity, and messages that remain in the queue for an extended period.

## Result

**SQS queue monitoring metrics were successfully reviewed using Amazon CloudWatch.**

## Conclusion

CloudWatch monitoring helps DevOps engineers observe SQS queue health and identify issues such as message backlogs, slow consumers, and processing failures.

## 2. Amazon MQ

## 2.1 Introduction to Amazon MQ

Amazon MQ is a managed message broker service provided by AWS.

Unlike SQS, Amazon MQ is designed around **traditional message-broker technologies**.

AWS supports managed broker engines including:

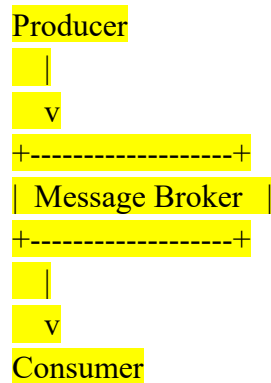
- ActiveMQ
- RabbitMQ

Amazon MQ is particularly useful when organizations already have applications that depend on traditional messaging protocols and APIs.

## 2.2 What is a Message Broker?

A message broker is software that receives, stores, routes and delivers messages between applications.

Example:



Amazon MQ provides the managed infrastructure for such a broker.

## 2.3 Amazon MQ vs SQS

This distinction is very important for interviews.

## SQS

Application



v

AWS SQS

AWS provides a queue service.

## Amazon MQ

Application



v

Amazon MQ Broker



v

Queue / Topic



v

Consumer

Amazon MQ provides a managed **message broker**.

## 2.4 Amazon MQ Engines

### ActiveMQ

ActiveMQ is a traditional enterprise messaging broker.

It can support technologies such as:

- JMS
- OpenWire
- AMQP
- MQTT
- STOMP

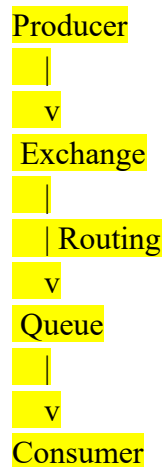
## RabbitMQ

RabbitMQ is a popular open-source message broker.

It commonly uses:

- AMQP
- Exchanges
- Queues
- Bindings
- Routing keys

Basic RabbitMQ flow:



## 2.5 Important Amazon MQ Components

### Broker

The actual Amazon MQ messaging server.

### Endpoint

The network endpoint through which applications communicate with the broker.

### Queue

Stores messages for consumers.

## Topic

Used for publish/subscribe messaging in supported broker technologies.

## Credentials

Used by applications/users to authenticate with the broker.

## Security Groups

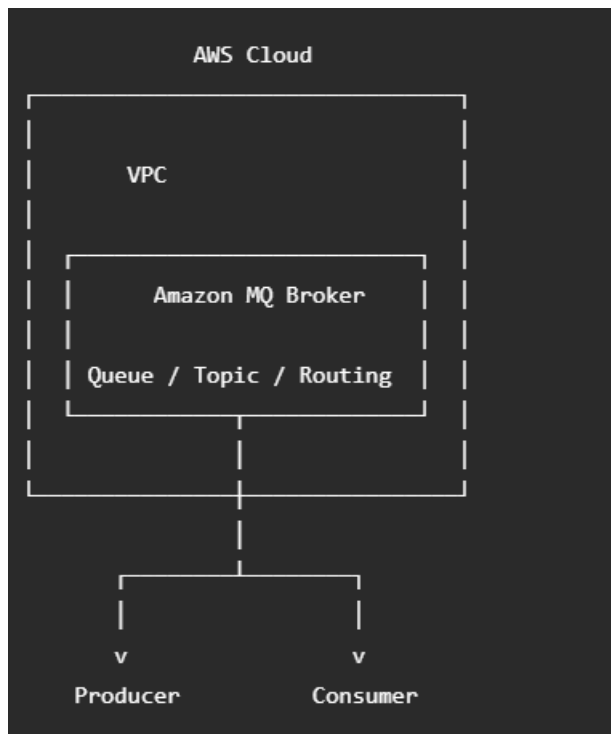
Control network access to the broker.

## VPC

Amazon MQ can operate within your AWS networking environment.

## 2.6 Amazon MQ Architecture

A typical architecture:



## 2.7 Amazon MQ Security

Important security components include:

- IAM where applicable
- Authentication
- Username/password
- Security Groups
- Network isolation
- Encryption
- TLS
- VPC configuration

For your documentation, explain:

The application must have network connectivity and valid authentication credentials to communicate with the broker.

## 2.8 Amazon MQ Real-Time Use Cases

### Legacy application migration

Existing Application

|

v

Existing Messaging Protocol

|

v

Amazon MQ

This is one of the most important use cases.

### Enterprise applications

Applications using established messaging technologies can use Amazon MQ without completely redesigning their messaging layer.

## Hybrid environments

On-Premises Application



Amazon MQ

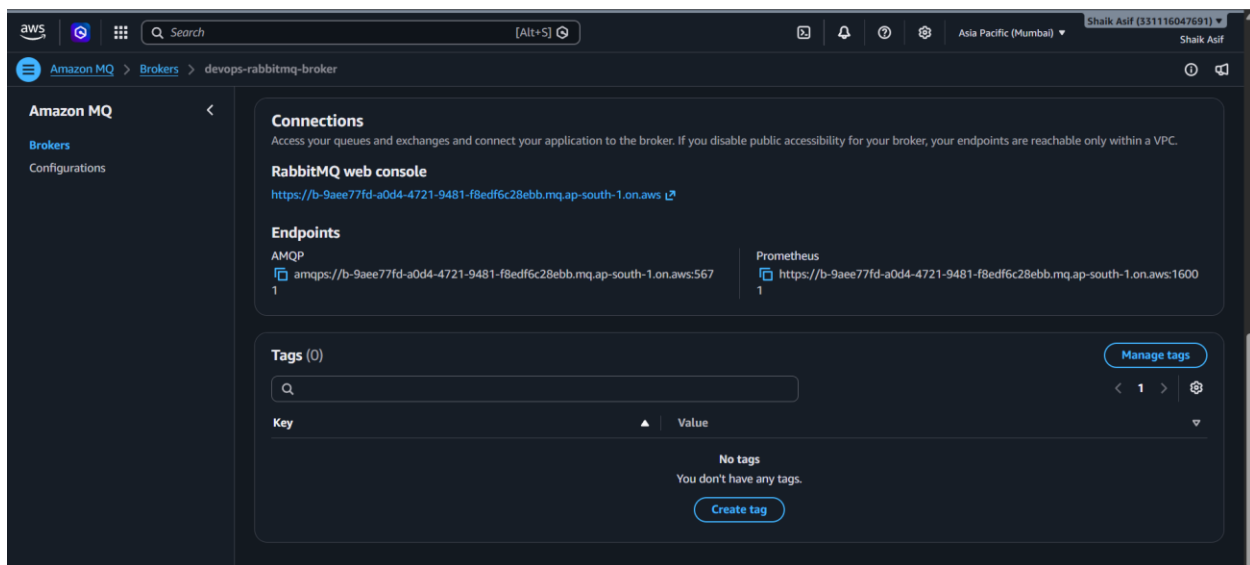
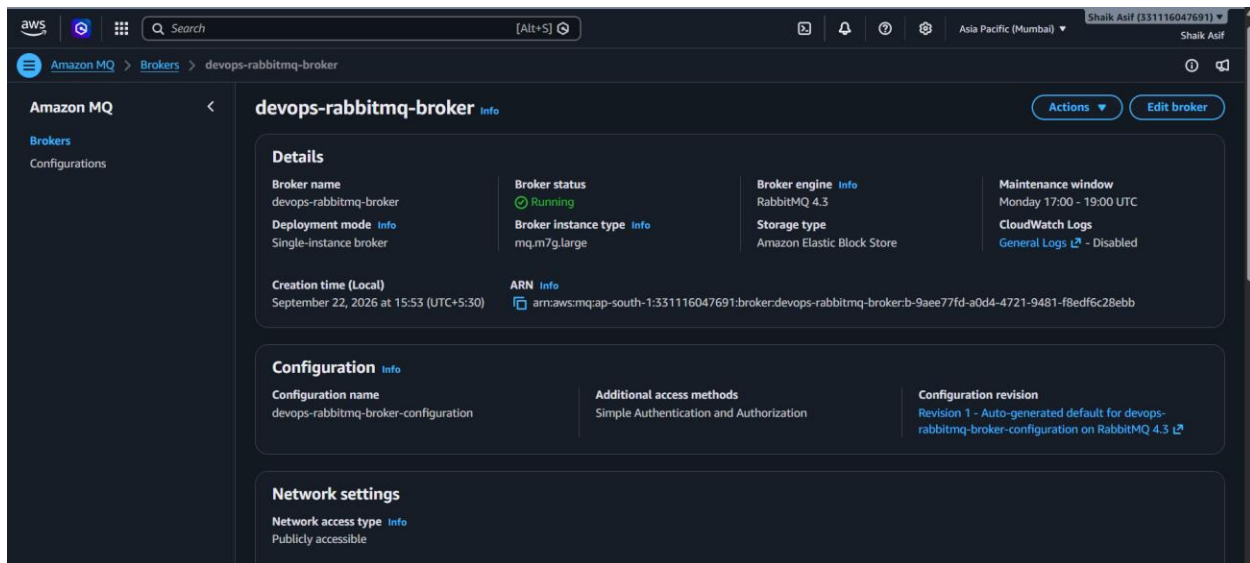


AWS Application

## Task 1 — Create an Amazon MQ RabbitMQ Broker

### Steps Used

1. Logged in to the AWS Management Console.
2. Selected the Mumbai (ap-south-1) region.
3. Opened Amazon MQ.
4. Navigated to Brokers.
5. Selected Create broker.
6. Selected RabbitMQ as the broker engine.
7. Selected a single-instance deployment for the practice environment.
8. Provided the broker name `devops-rabbitmq-broker`.
9. Selected a suitable low-cost broker instance.
10. Configured administrator credentials.
11. Configured the VPC, subnet and security settings.
12. Created the RabbitMQ broker.
13. Verified that the broker reached the **Running** state.



## Observation

Amazon MQ created a managed RabbitMQ broker inside the selected AWS networking environment. AWS manages the underlying broker infrastructure while we configure and use the messaging system.

## Result

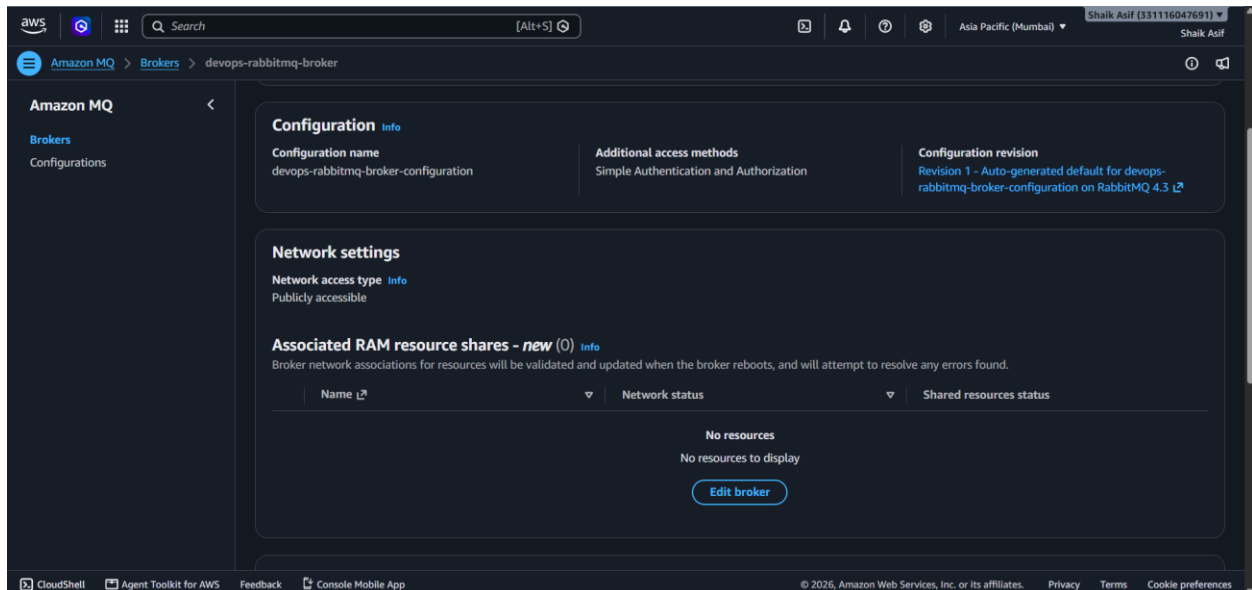
The devops-rabbitmq-broker RabbitMQ broker was successfully created.

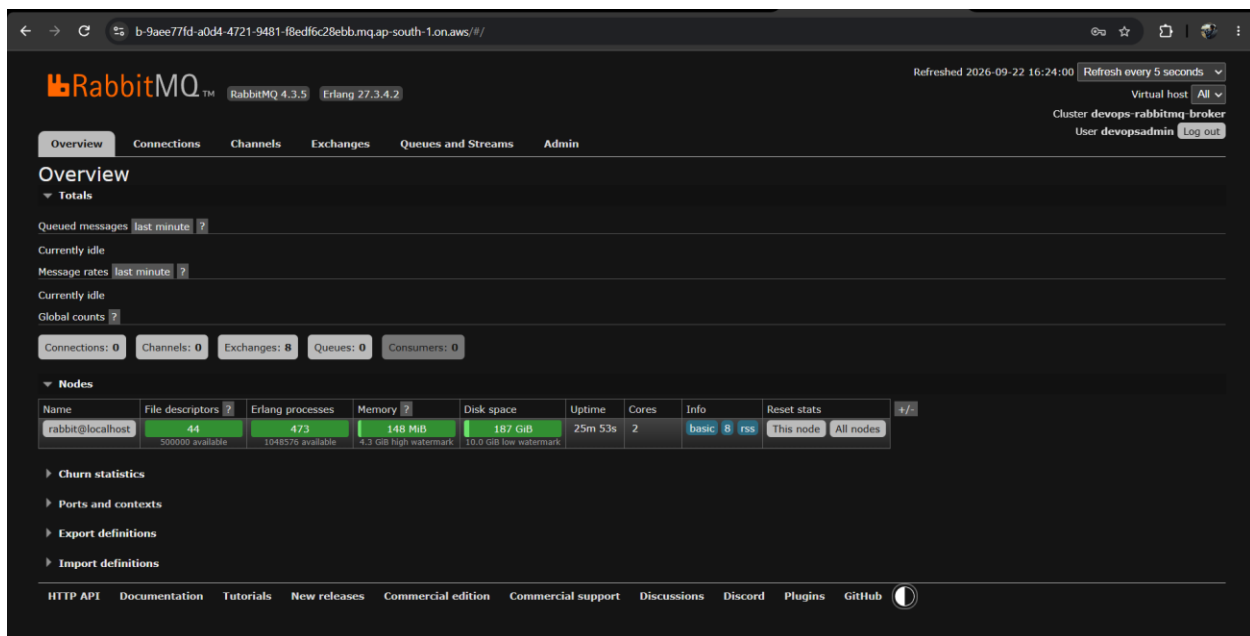
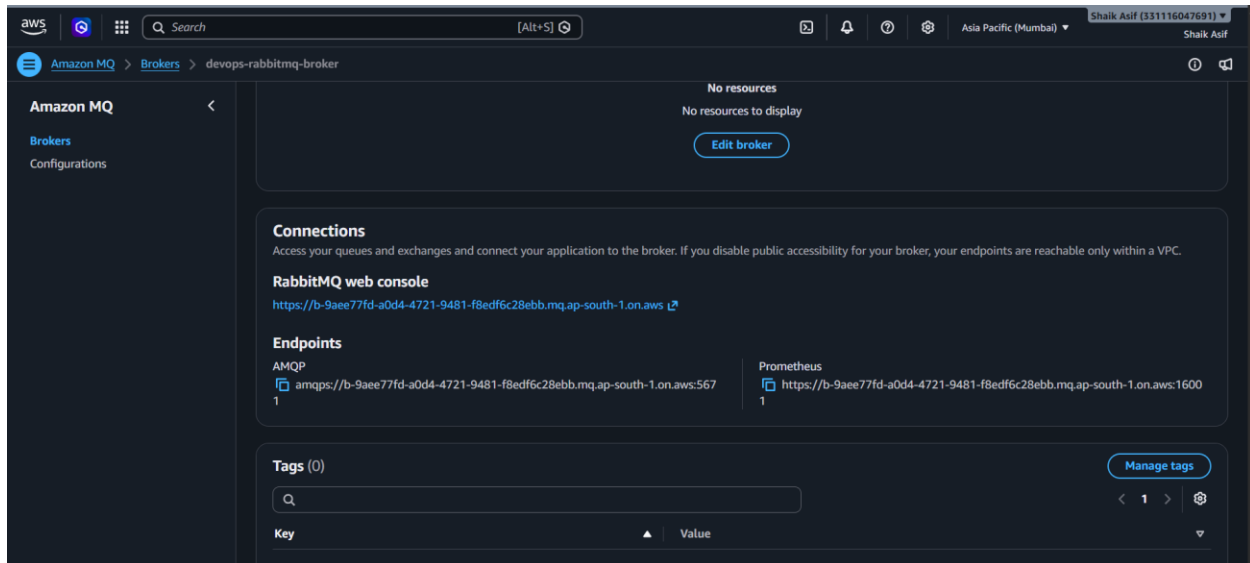
## Conclusion

Amazon MQ provides a managed message-broker environment without requiring us to manually install and maintain RabbitMQ infrastructure.

## Task 2 — Amazon MQ Networking, Credentials & Security

- Opened the Amazon MQ service.
- Opened devops-rabbitmq-broker.
- Verified the broker status as **Running**.
- Reviewed the broker deployment mode and instance type.
- Reviewed the **Network settings**.
- Verified **Publicly accessible** network access.
- Reviewed the **RabbitMQ Web Console** connection.
- Opened the RabbitMQ Web Console using the configured credentials.
- Reviewed the **AMQPS endpoint**.
- Verified secure AMQP connectivity through **port 5671**.
- Reviewed the **Prometheus endpoint** on port 16001.
- Reviewed the broker's available authentication method: **Simple Authentication and Authorization**.





## Observation

The RabbitMQ broker is running as a single-instance Amazon MQ broker and is publicly accessible. Amazon MQ provides a RabbitMQ Web Console, secure AMQPS endpoint on port 5671, and Prometheus endpoint for monitoring.

## Result

Amazon MQ networking and connectivity settings were successfully reviewed.

## Task 3 — RabbitMQ Exchanges, Queues & Routing

- Opened the RabbitMQ Web Console.
- Navigated to **Exchanges** and reviewed the available exchanges.
- Navigated to **Queues and Streams**.
- Created a queue named devops-task-queue.
- Opened the queue configuration.
- Reviewed the queue's binding configuration.
- Created a binding from the amq.direct exchange.
- Configured the routing key as devops.
- Verified the exchange-to-queue binding.

RabbitMQ 4.3.5 Erlang 27.3.4.2

Refreshed 2026-09-22 16:34:01 Refresh every 5 seconds

Virtual host All

Cluster devops-rabbitmq broker

User devopsadmin Log out

Overview Connections Channels Exchanges **Queues and Streams** Admin

### Queues

▼ All queues (1)

Pagination

Page 1 of 1 - Filter:  ☐ Regex

Displaying 1 item, page size up to: 100

| Overview     |                   |        |           |         |  | Messages |         |       | Message rates |         |     |     |
|--------------|-------------------|--------|-----------|---------|--|----------|---------|-------|---------------|---------|-----|-----|
| Virtual host | Name              | Type   | Features  | State   |  | Ready    | Unacked | Total | incoming      | deliver | get | ack |
| /            | devops-task-queue | quorum | D DR Args | running |  | 0        | 0       | 0     |               |         |     |     |

▼ Add a new queue

Type: Default for virtual host

Virtual host: /

Name:

Durability: Durable

Arguments:  =  String

Add Auto expire Message TTL Overflow behaviour

Single active consumer Dead letter exchange Dead letter routing key

Max length Max length bytes Maximum priority Leader locator

Add queue

HTTP API Documentation Tutorials New releases Commercial edition Commercial support Discussions Discord Plugins GitHub

RabbitMQ 4.3.5 Erlang 27.3.4.2

Refreshed 2026-09-22 16:36:11 Refresh every 5 seconds

Virtual host All

Cluster devops-rabbitmq broker

User devopsadmin Log out

Overview Connections Channels **Exchanges** Queues and Streams Admin

### Exchange: amq.direct

▼ Overview

Message rates last minute

Currently idle

Details

Type direct

Features durable: true

Policy

▼ Bindings

This exchange

↓

| To                | Routing key | Arguments |        |
|-------------------|-------------|-----------|--------|
| devops-task-queue | devops      |           | Unbind |

Add binding from this exchange

To queue:

Routing key:

Arguments:  =  String

Bind

## Observation

RabbitMQ uses exchanges to receive messages from producers and route them to queues using routing rules. The devops-task-queue queue was connected to the amq.direct exchange using the devops routing key.

## Result

**A RabbitMQ queue and exchange-to-queue binding were successfully configured.**

## Conclusion

RabbitMQ provides flexible message routing through **exchanges, queues, bindings, and routing keys**. This allows messages to be routed to specific queues based on application requirements.

## Task 4 — Send and Receive a RabbitMQ Message

- Opened the RabbitMQ Web Console.
- Navigated to the amq.direct exchange.
- Used the **Publish Message** option.
- Configured the routing key as devops.
- Entered the message Hello from Amazon MQ RabbitMQ Task 4.
- Published the message.
- Opened the devops-task-queue.
- Verified that the message was available in the queue.
- Retrieved the message using the **Get Message(s)** option.
- Verified the message body.
- Acknowledged/removed the processed message.
- Verified that the message was no longer waiting in the queue.

RabbitMQ™

RabbitMQ 4.3.5

Erlang 27.3.4.2

Refreshed 2026-09-22 16:40:35

Refresh every 5 seconds

Virtual hostAll

Cluster devops-rabbitmq-broker

User devopsadmin

Log out

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Routing key:

Arguments:

=

String

Bind

▼ Publish message

Routing key:devops

Headers: ?=String

Properties: ?=

Payload:Hello from Amazon MQ RabbitMQ Task 4

Payload encoding:String (default)

Publish message

▶ Delete this exchange

HTTP API

Documentation

Tutorials

New releases

Commercial edition

Commercial support

Discussions

Discord

Plugins

GitHub

RabbitMQ™

RabbitMQ 4.3.5

Erlang 27.3.4.2

Refreshed 2026-09-22 16:42:38

Refresh every 5 seconds

Virtual hostAll

Cluster devops-rabbitmq-broker

User devopsadmin

Log out

Overview

Connections

Channels

Exchanges

Queues and Streams

Admin

Queues

▼ All queues (1)

Pagination

Page 1 of 1 - Filter:Regex

Displaying 1 item , page size up to: 100

| Overview     |                   |        |                                                  |         |  | Messages |         |       |  | Message rates |         |     |     | +/- |
|--------------|-------------------|--------|--------------------------------------------------|---------|--|----------|---------|-------|--|---------------|---------|-----|-----|-----|
| Virtual host | Name              | Type   | Features                                         | State   |  | Ready    | Unacked | Total |  | incoming      | deliver | get | ack |     |
| /            | devops-task-queue | quorum | <span>0</span> <span>DR</span> <span>Args</span> | running |  | 1        | 0       | 1     |  | 0.00/s        |         |     |     |     |

▼ Add a new queue

Type:Default for virtual host

Virtual host:/

Name:

Durability:Durable

Arguments: =String

Add

Auto expire | Message TTL | Overflow behaviour

Single active consumer | Dead letter exchange | Dead letter routing key

Max length | Max length bytes | Maximum priority | Leader locator

Add queue

HTTP API

Documentation

Tutorials

New releases

Commercial edition

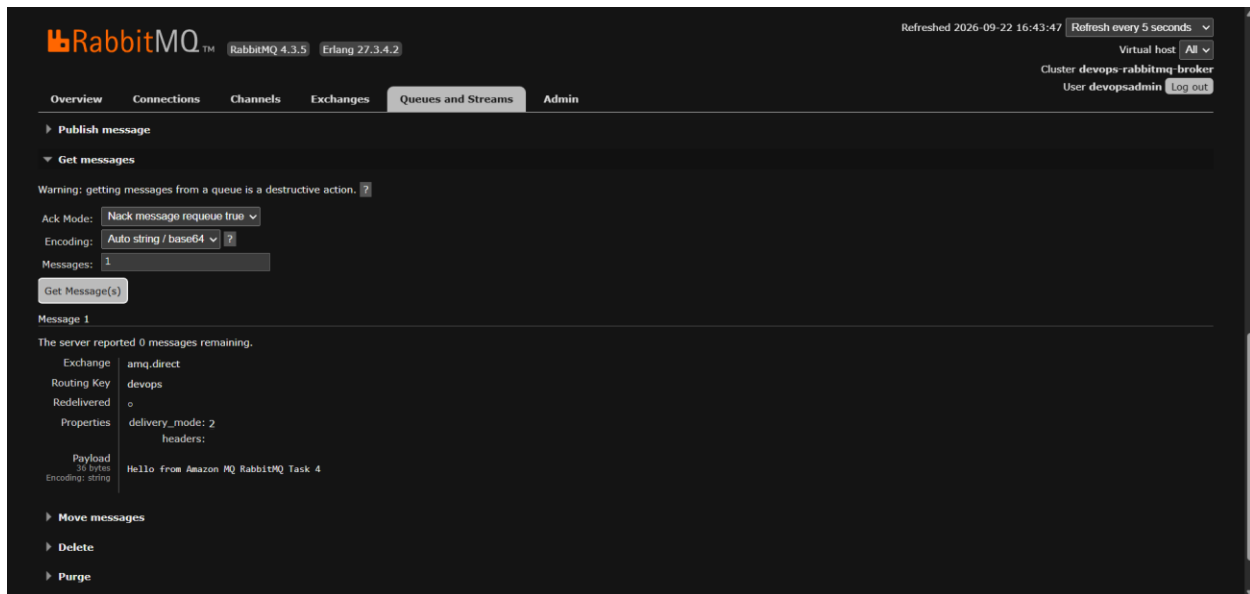
Commercial support

Discussions

Discord

Plugins

GitHub



## Observation

The message was published to the `amq.direct` exchange using the `devops` routing key. The exchange routed the message to `devops-task-queue`, where it was successfully retrieved by the consumer.

## Result

The RabbitMQ message was successfully published, routed, received, and acknowledged.

## Conclusion

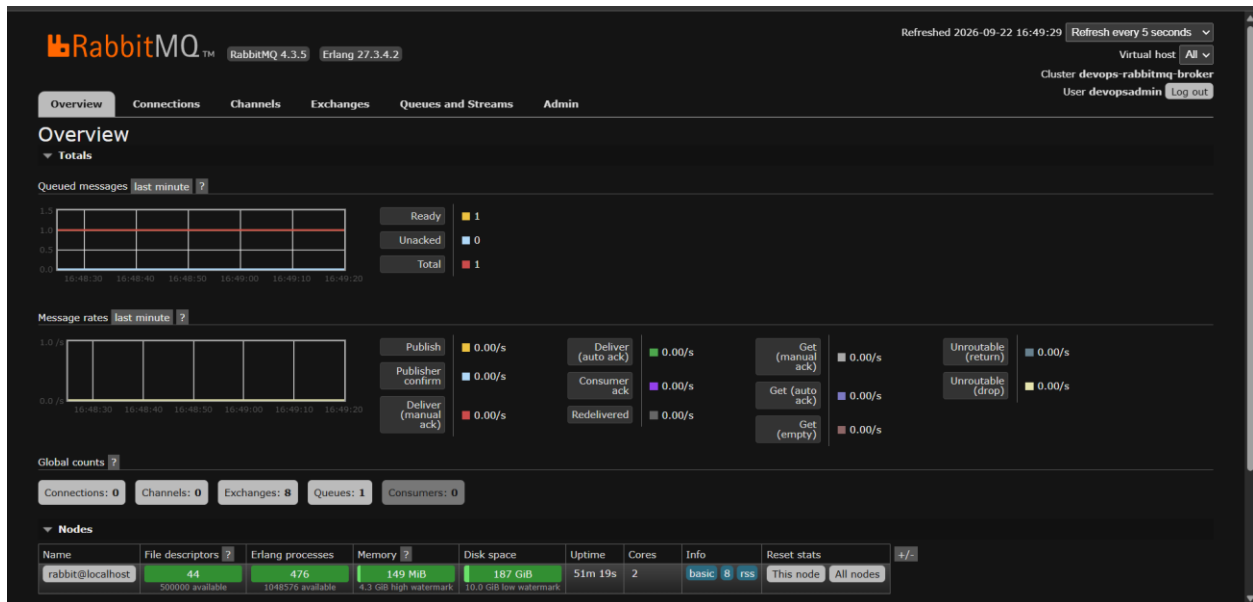
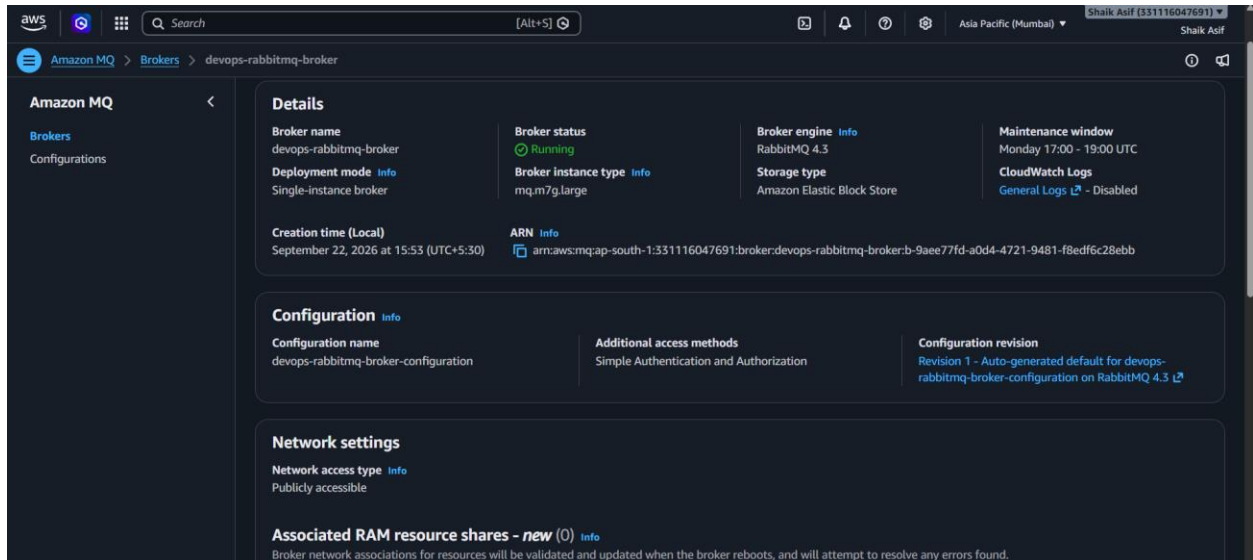
RabbitMQ uses **exchanges, routing keys, bindings, and queues** to control how messages move from producers to consumers. This provides more flexible routing capabilities than a simple queue-based messaging model.

## Task 5 — Amazon MQ Monitoring & Cleanup

### Steps Used

1. Opened Amazon MQ in the AWS Management Console.
2. Opened the `devops-rabbitmq-broker`.
3. Verified that the broker status was **Running**.
4. Verified the RabbitMQ engine and single-instance deployment.

5. Opened the RabbitMQ Web Console.
6. Reviewed broker overview information.
7. Checked the configured queue devops-task-queue.
8. Verified the queue/message status after completing the message test.



## Observation

The Amazon MQ RabbitMQ broker was running successfully. The RabbitMQ Web Console provided visibility into queues, exchanges, connections, and message activity.

## Result

The broker health and RabbitMQ resources were successfully verified.

## Conclusion

Amazon MQ provides a **managed RabbitMQ broker**, while RabbitMQ's management console can be used to monitor queues, exchanges, connections, and message activity.

## 3. AWS Secrets Manager

### 3.1 Introduction

AWS Secrets Manager is a managed service for securely storing and managing sensitive information.

Examples:

Database Password

API Key

Username

Password

OAuth Token

Application Credential

Instead of putting secrets directly into source code, applications can retrieve them when required.

### 3.2 Problem with Hard-Coded Credentials

Bad implementation:

```
username = "admin"
```

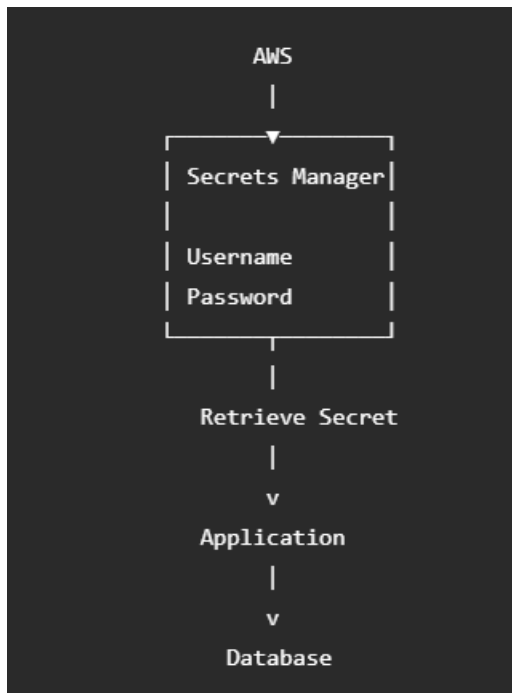
```
password = "MyPassword123"
```

If this code is pushed to GitHub:



This creates a security problem.

### 3.3 Better Architecture



The application retrieves the secret when needed rather than storing the password directly in its code.

### 3.4 Secret Components

A secret can contain:

```
{  
  "username": "devopsuser",  
  "password": "*****"  
}
```

AWS assigns the secret an ARN.

Example structure:

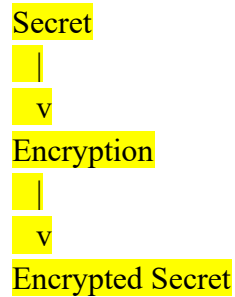
arn:aws:secretsmanager:region:account-id:secret:name

### 3.5 Secret Encryption

Secrets Manager encrypts secret values.

AWS Key Management Service (KMS) can be involved in encryption management.

Conceptually:

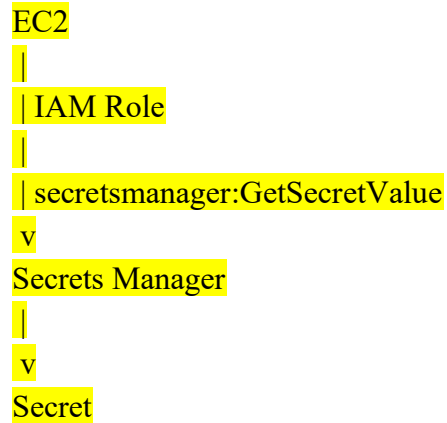


Only authorized identities should be able to retrieve the secret.

### 3.6 IAM and Secrets Manager

This is an important DevOps concept.

Suppose an EC2 instance needs a database password.



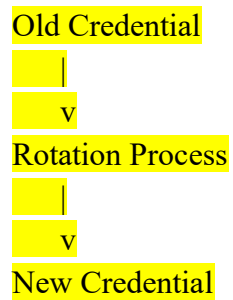
The IAM role determines whether the EC2 application can retrieve the secret.

### 3.7 Secret Rotation

Credentials shouldn't necessarily remain unchanged forever.

Secrets Manager supports rotation for supported configurations.

Conceptually:

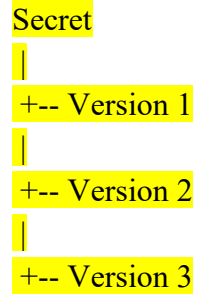


This helps reduce the risk associated with long-lived credentials.

### 3.8 Secret Versioning

Secrets Manager can maintain different versions of a secret during the secret lifecycle.

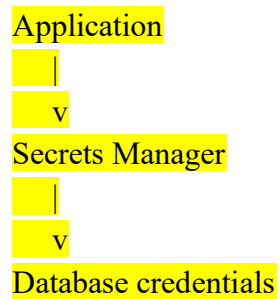
Conceptually:



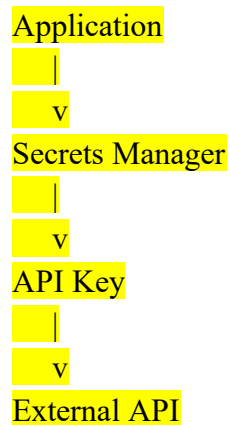
Version staging labels can identify the current version during rotation workflows.

### 3.9 Secrets Manager Real-Time Use Cases

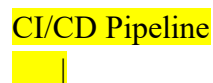
#### Database credentials

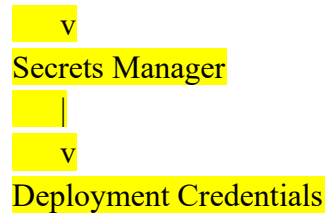


#### API keys



#### DevOps pipelines





## Application configuration

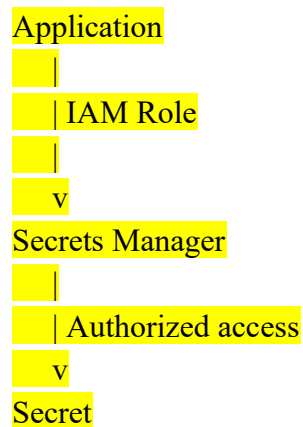
Sensitive configuration can be stored separately from application source code.

### 3.10 Secrets Manager Security

Important areas:

- IAM
- KMS encryption
- Resource policies where applicable
- Rotation
- Least privilege
- CloudTrail auditing
- Secure retrieval

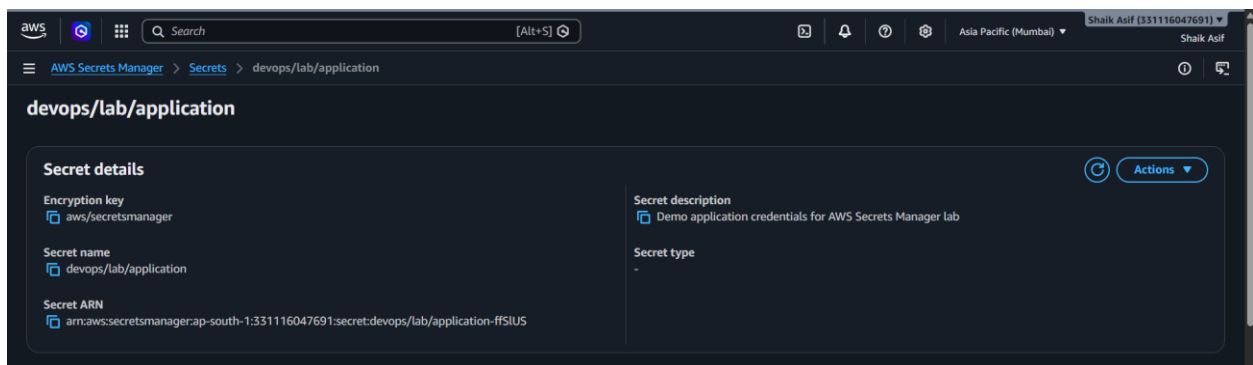
A good security model is:



## Task 1: Create and Store a Secret

### Steps Used

1. Opened AWS Secrets Manager.
2. Selected the Mumbai (ap-south-1) region.
3. Selected **Store a new secret**.
4. Selected **Other type of secret**.
5. Created username and password key-value pairs using dummy credentials.
6. Used the default AWS-managed encryption option.
7. Configured the secret name as `devops/lab/application`.
8. Added a description.
9. Kept automatic rotation disabled for the initial lab.
10. Stored the secret.
11. Verified that the secret was successfully created.



## Observation

The credentials were stored as a secret in AWS Secrets Manager instead of being directly placed inside an application configuration.

## Result

The secret devops/lab/application was successfully created and stored.

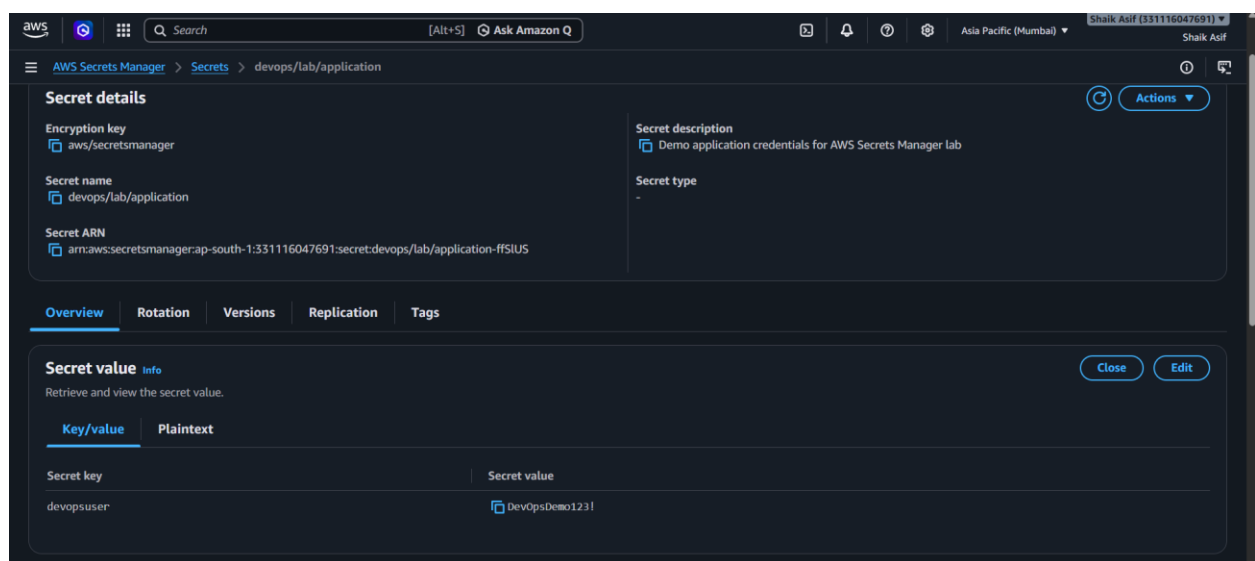
## Conclusion

AWS Secrets Manager provides a centralized and secure way to store sensitive information such as passwords, API keys, and database credentials. Applications can retrieve these secrets when required instead of hardcoding sensitive values in application code.

## Task 2 — Retrieve the Secret

### Steps Used

1. Opened AWS Secrets Manager.
2. Navigated to **Secrets**.
3. Opened the secret devops/lab/application.
4. Selected **Retrieve secret value**.
5. Verified that the stored key/value information could be retrieved.
6. Reviewed the secret metadata and version information.



## Observation

The secret value stored in Secrets Manager can be retrieved when required. Access to the secret can be controlled through IAM permissions.

## Result

The previously stored secret was successfully retrieved from AWS Secrets Manager.

## Conclusion

Secrets Manager allows applications and authorized users to retrieve sensitive information securely without storing credentials directly in application code.

## Task 3 — Retrieve Secret Using AWS CLI

### Steps Used:

1. Created/configured the Asifshaik IAM user for controlled secret retrieval.
2. Generated new CLI credentials for the IAM user.
3. Configured AWS CLI using `aws configure`.
4. Set the region to `ap-south-1`.
5. Verified the IAM identity using `aws sts get-caller-identity`.
6. Retrieved `devops/lab/application` using `aws secretsmanager get-secret-value`.
7. Verified that the secret was successfully returned.

```
asifs@Asif_Shaik MINGW64 ~
$ aws sts get-caller-identity
{
  "UserId": "AIDAU2GAN5VFVOC7PDQUI",
  "Account": "331116047691",
  "Arn": "arn:aws:iam::331116047691:user/Asifshaik"
}

asifs@Asif_Shaik MINGW64 ~
$ aws secretsmanager get-secret-value \
> --secret-id devops/lab/application \
> --region ap-south-1
{
  "ARN": "arn:aws:secretsmanager:ap-south-1:331116047691:secret:devops/lab/app
lication-ffSLUS",
  "Name": "devops/lab/application",
  "VersionId": "b50a022c-2f01-401c-ac7f-888f89266a8f",
  "SecretString": "{\"devopsuser\":\"DevOpsDemo123!\"}",
  "VersionStages": [
    "AWSCURRENT"
  ],
  "CreateDate": "2026-09-22T17:24:30.894000+05:30"
}
```

### Observation:

The IAM user was able to retrieve the secret through AWS CLI using the `secretsmanager:GetSecretValue` permission.

**Result:**

The secret was successfully retrieved programmatically using AWS CLI.

**Conclusion:**

AWS Secrets Manager can be integrated into DevOps scripts and automation, while IAM permissions control who can retrieve sensitive information.